Bangalore, Karnataka
phone on request
im.swarnabha2001@gmail.com

# Swarnabha Das

Backend Software Developer

linkedin.com/in/swarnabhadas
github.com/sd2001
sd2001.github.io

## Work Experience

**Software Engineer 2 (SE-1 → SE-2, promoted Feb 2026)** **January 2025 — Present**
**Parspec** - AI platform digitizing building-product data for electrical distributors, manufacturers & contractors (Bangalore)

### Crawler Pipeline Rebuild

- Rebuilt the crawler that pulls **3M manufacturer datasheets** off the web every 2 to 3 weeks, a cycle that has to finish inside **14 days** before the next one starts. Moved pipeline state out of Jira ticket statuses across 5,000 manufacturers into **Postgres** and made it **event-driven**, tracking each PDF rather than whole manufacturers, so one bad file no longer forces a 10,000-product rerun.
- Re-platformed from fixed **EC2** nodes, where each stage sat on its own instances and a slow stage meant manually adding machines, onto **EKS** so stages scale on their own and we spend only where a stage needs it; sizing on **AWS Graviton** modelled **~35% lower** cost ($2,657 to $1,718/mo).

### Datasheet Search & Ingestion (~1.4M product families)

- Built the pipeline turning manufacturer PDFs into a searchable catalogue across 3 **Qdrant** collections. Each page routes to the cheapest reader that handles it: plain text first, **Docling/TableFormer** on GPU for tables, and only hard pages reach a paid vision model (**Qwen3-VL-32B**, then **Gemini 2.5 Flash**), so spend tracks difficulty, not count. Extraction runs at **0.906 macro-F1**, every PDF keyed by hash so reruns repair, never duplicate.
- Made part-number search typo tolerant by indexing each code as overlapping **3-character chunks**, so a search for `LC1D18B7` still finds `LC1D18–B7` where ordinary keyword search returns nothing, with a similarity floor that abstains instead of guessing. Also closed a hole where a datasheet that failed extraction vanished with no retry and no record of it.

### Inventory Search (8.4M products, 100s of distributors)

- Built the search behind Parspec's Crossing product on one Qdrant collection. Every query runs three searches at once, a 768-dim **all-mpnet-base-v2** semantic vector plus two BM25 keyword vectors, merged with **Reciprocal Rank Fusion**. Every distributor gets its own **HNSW search graph** inside that single index, so catalogues stay isolated without filtering afterwards, and **int8** compression with rescoring cut vector memory to **~25%** at the same quality.
- Re-ranked the top 50 hits on GPU with an **ms-marco-MiniLM-L6-v2** cross-encoder and added an **LLM category classifier** (Gemini, GPT-5-nano) that runs alongside the search so it costs no extra time. Also translated the shorthand electricians search with into the wording distributors store, so "16A 1 pole" matches products described a dozen different ways and every spelling of a part number lands on one product. Grouping by product then shows a customer their true stock across every warehouse instead of one row per warehouse.
- Migrated **7.1M live inventory records** from DynamoDB into Qdrant with AI enrichment on each record, and keep the two in sync within seconds through **change-data-capture**: DynamoDB Streams to **Lambda**, Lambda to **SQS**, and SQS drained by **EKS workers** that enrich, embed and write into Qdrant; a **~73%** prompt cache made incident replays near free.

### GPU Serving, Performance & Cloud Cost

- Moved all GPU inference from **Tesla T4** onto **NVIDIA L4** (**g6**) instances and converted the models to **TensorRT/ONNX**, which raised **throughput** enough to keep full **FP32** precision instead of dropping to FP16 for latency: the BERT category model now serves **40 req/s per pod** where it saturated past 15, and text embedding holds **sub-100ms p95 at 30 req/s** against 350ms at 15.
- Cut p95 text extraction **18.5s to 0.95s (95% faster)**. The tail was memory fragmentation and **OOM kills**, not parsing, so I added locking around the shared **PyMuPDF (fitz)** handles to stop the multithreaded path segfaulting and getting killed (exit 137), and moved the service to **c8i** instances.
- **Cloud cost, ~$2.6K/month:** a **~$5,000/mo** NAT bill with no attribution, so we tagged every service at module level and read **VPC flow logs** against those tags until every dollar had an owner. Moved S3, DynamoDB, CloudWatch and SNS onto **private endpoints**, keeping traffic inside AWS (**30-40%**), re-homed chatty services into the account calling them (**$560/mo**) and killed cross-AZ gateway hops (**~$2,100/mo**).
- Tech stack: Python, FastAPI, PostgreSQL, DynamoDB, Qdrant, OpenSearch, Valkey, SQS/SNS, Lambda, EKS, PyTorch, TensorRT, Gemini/Qwen-VL

**Software Development Engineer - 1 (Backend and Devops)** **January 2023 — December 2024**
**Pibit.ai (YC W21)** - Unlock Insightful Information from vastly unstructured insurance/underwriting data) (Bangalore)

- Integrated **Lossrun** (Insurance LOB), optimised its core logical components on our production server and owned it thereafter, running the infra and **CI/CD on AWS**; maintained and optimised the insurance **data extraction pipeline by 18%** and set up **analytics for logging & error tracebacks**.
- Introduced **multi-stage docker builds** for faster and smaller builds & improved **K8s** pod performance using **graceful deployments**.
- Built an **e2e in-house analytics driven testing framework (PiEval)** from scratch to automate test & logic coverage reports of templates & APIs after deployments, which halted manual operations-driven testing and **saved up to 3 days** before production & client releases.
- Tech stack: Python, FastAPI, DynamoDB, PostgreSQL, AWS SQS, AWS S3, AWS Lambda, Kubernetes, AWS Cloudformation

**Research Intern** **June 2022 — September 2022**
**Mitacs GRI 2022** - Researched on a code vulnerability project under Prof. Shaowei Wang at University of Manitoba, Winnipeg (Canada)

- Assessing the robustness of **deep-learning vulnerability detection** via **code augmentation** and analyzing the flip ratio
- Analyzing performance metrics by **program transformation** techniques inspired by **Paper 1** & **Paper 2**

## Technical Skills

| | |
|---|---|
| **Languages** | Python(Proficient), C++, GoLang, SQL |
| **Libs/Frameworks** | FastAPI, Flask, Django, PyTorch, TensorRT/ONNX, sentence-transformers, PyMuPDF, asyncio, SQLAlchemy |
| **Databases/Search** | PostgreSQL, DynamoDB, MySQL, MongoDB, Redis/Valkey, **Qdrant** (HNSW, hybrid dense+sparse), OpenSearch |
| **AI/ML** | RAG, vector search, embeddings, cross-encoder reranking, LLMs (Gemini, Qwen-VL, OpenRouter), BERT |
| **Cloud/DevOps** | AWS (EKS, Lambda, SQS/SNS, S3, VPC/PrivateLink, CloudFormation), Kubernetes, Docker, CI/CD, GPU (L4/T4) |

## Education

**Bachelor of Technology, Computer Science,** SRM Institute of Science and Technology, GPA: 9.47/10 **2019 — 2023**

## Publications

**Optimizing Apache Kafka Deployments - Configuration customization is all you need,** **Jan 2023 — June 2023**
IEEE - tuning Kafka around the 4 major pillars of Latency, Throughput, Durability and Availability